HELLA company 업무관리사이트 — NFC 반출/반입 시스템 상세 설계서

문서 ID: NFC_CHECKOUT_CHECKIN_ENGINE 문서 목적: HELLA company 업무관리사이트의 NFC 기반 장비 반출/반입 기능을 실제 구현 가능한 수준으로 상세 설계한다.

이 문서는 다음의 기준 문서다. - NFC 이벤트 처리 흐름 - 장비 상태 머신 - 반출/반입 트랜잭션 규칙 - API 구조 - 실시간 처리 방식 - 예외 상황 처리 기준 - 로그 및 감사 정책 - 향후 확장 구조

이 문서는 단순 UI 문서가 아니라 **장비 흐름 엔진 설계 문서**다.

---

1. 시스템 개요

1-1. 시스템 목적

이 시스템은 회사 장비에 부착된 NFC 태그를 통해 다음을 정확하게 수행하는 것을 목표로 한다.

1. 장비 식별
2. 장비와 사용자 연결
3. 반출 기록 저장
4. 반입 기록 저장
5. 현재 장비 상태 추적
6. 미반입 장비 자동 식별
7. 장비 이력 조회
8. 관리자 통제 및 감사 추적

1-2. 운영 전제

- 사용자 기기: NFC 지원 안드로이드 스마트폰
- 실행 환경: 모바일 웹앱
- 브라우저: Chrome 계열 권장
- 네트워크: 온라인 기본, 오프라인은 차기 확장
- 장비 1개 ↔ NFC 태그 1개 1:1 매핑 원칙

1-3. 핵심 원칙

- 장비 상태는 항상 서버 DB 기준으로 판정한다.
- NFC 태그는 식별 키일 뿐, 실질 상태의 기준은 DB다.
- 반출/반입은 UI 버튼이 아니라 **상태 전이 트랜잭션**이다.
- 장비 상태 변경과 로그 생성은 반드시 한 묶음으로 처리한다.

---

2. 핵심 엔티티

2-1. Equipment

장비의 현재 상태를 가지는 주체

핵심 필드 - equipmentId - equipmentCode - name - status - currentHolderUserId - currentHolderName - nfcTagId - lastCheckoutAt - lastCheckinAt - isActive

2-2. NfcTagMapping

NFC 태그와 장비의 연결 정보

핵심 필드 - nfcTagId - equipmentId - status - mappedAt

2-3. EquipmentLog

장비 상태 변화의 이력

핵심 필드 - equipmentId - actionType - actorUserId - performedAt - result - reason - useLocation - note

2-4. User

반출/반입 행위 주체

핵심 필드 - userId - role - status

---

3. 장비 상태 머신

3-1. 상태 정의

AVAILABLE

- 보관 중
- 반출 가능
- 현재 보유자 없음

CHECKED_OUT

- 반출 중
- 반입 가능
- 현재 보유자 존재

OVERDUE

- 반출 후 기준 시간 초과

- 관리자 주의 필요
- 반입 가능
- 현재 보유자 존재

DISABLED

- 사용 중지
- 반출/반입 일반 처리 불가
- 관리자만 상태 변경 가능

3-2. 상태 전이 다이어그램

```
AVAILABLE
  └─(checkout)→ CHECKED_OUT

CHECKED_OUT
  └─(elapsed timeout)→ OVERDUE

CHECKED_OUT
  └─(checkin)→ AVAILABLE

OVERDUE
  └─(checkin)→ AVAILABLE

AVAILABLE
  └─(admin disable)→ DISABLED

CHECKED_OUT / OVERDUE
  └─(admin force disable)→ DISABLED   [권장: 예외 처리]

DISABLED
  └─(admin enable)→ AVAILABLE
```

3-3. 상태 전이 규칙

AVAILABLE → CHECKED_OUT

조건 - 장비 활성 상태 - NFC 매핑 존재 - 사용자 ACTIVE - 현재 상태 AVAILABLE

처리 - currentHolderUserId 설정 - currentHolderName 설정 - lastCheckoutAt 기록 - 로그 생성

CHECKED_OUT → AVAILABLE

조건 - 현재 상태 CHECKED_OUT - 반입 수행 사용자 인증 성공

처리 - currentHolderUserId 제거 - currentHolderName 제거 - lastCheckinAt 기록 - 로그 생성

CHECKED_OUT → OVERDUE

조건 - lastCheckoutAt 기준 일정 시간 초과 - 아직 반입 안 됨

처리 - 상태만 OVERDUE 전환 - 알림/강조 표시 - 로그 생성 여부는 정책 선택

OVERDUE → AVAILABLE

조건 - 정상 반입 완료

처리 - 보유자 제거 - lastCheckinAt 기록 - 로그 생성

ANY → DISABLED

조건 - 관리자 수행

처리 - 일반 사용자 흐름 차단 - 감사 로그 필수

---

4. NFC 이벤트 처리 흐름

4-1. 전체 처리 개요

NFC 태깅은 단순 UID 읽기가 아니라 아래 전체 흐름을 말한다.

```
NFC 태그 접촉
→ UID 읽기
→ UID 유효성 확인
→ 매핑 장비 조회
→ 사용자 상태 확인
→ 장비 상태 조회
→ 현재 화면 모드 확인
→ 반출/반입 가능성 판단
→ 트랜잭션 실행
→ 로그 저장
→ UI 결과 표시
```

4-2. 공통 사전 검증

Step 1. Web NFC 지원 여부 확인

실패 시 - "이 기기는 NFC를 지원하지 않습니다." 표시 - 대체 수동 선택 UI로 이동 여부 정책 선택

Step 2. 브라우저 권한 확인

실패 시 - 권한 요청 - 거부 시 안내 문구 표시

Step 3. 로그인 사용자 확인

실패 시 - 로그인 화면 이동

Step 4. 사용자 상태 확인

조건 - ACTIVE만 허용

실패 시 - 접근 차단

4-3. NFC UID 읽기 단계

입력

- 단말에 접촉된 NFC 태그

출력

- nfcTagId

규칙

- UID를 문자열로 정규화
- 대소문자/구분자 포맷 일관화

예 - `04AABBCC1122`

실패 처리

- 읽기 실패 → 재시도 안내
- 불완전 읽기 → 재태깅 유도

4-4. 매핑 장비 조회 단계

입력

- nfcTagId

처리

- `nfc_tag_mappings`에서 ACTIVE 매핑 조회
- 대응 `equipmentId` 조회
- `equipments` 문서 조회

실패 처리

태그 미등록

- 관리자: 등록 유도 가능

- 사원: 처리 차단 + "등록되지 않은 장비 태그" 표시

중복 ACTIVE 매핑

- 시스템 오류 상태
- 관리자 알림
- 사용자 처리 차단

4-5. 모드 기반 처리 방식

NFC 처리에는 두 가지 방식이 있다.

방식 A. 명시적 모드 방식

사용자가 먼저 선택 - 반출 모드 - 반입 모드

장점 - 실수 적음 - 구현 명확

방식 B. 자동 판정 방식

장비 상태에 따라 자동 판정 - AVAILABLE → 반출로 판단 - CHECKED_OUT / OVERDUE → 반입으로 판단

장점 - 빠름 단점 - 현장 실수 가능성 있음

권장

MVP는 **명시적 모드 방식** 우선. 차기 확장에서 자동 판정 추가 가능.

---

5. 반출 처리 엔진

5-1. 반출 입력값

- actorUserId
- nfcTagId
- equipmentId
- reason
- useLocation
- note
- clientTimestamp(optional)

5-2. 반출 전 검증

사용자 검증

- 로그인 상태
- ACTIVE 상태

- 권한 유효

장비 검증

- equipment exists
- isActive = true
- status = AVAILABLE
- nfcTagId 일치

동시성 검증

- 최신 장비 상태 재확인
- 이미 다른 요청으로 반출 중이면 차단

5-3. 반출 트랜잭션 처리

트랜잭션 안에서 반드시 함께 수행

1. equipment 문서 재조회
2. status가 AVAILABLE인지 다시 확인
3. status를 CHECKED_OUT으로 변경
4. currentHolderUserId 설정
5. currentHolderName 설정
6. lastCheckoutAt = serverTimestamp
7. equipment_logs 문서 생성
8. audit_logs 필요 시 생성
9. 커밋

5-4. 반출 성공 결과

UI 표시 - 장비명 - 반출 성공 메시지 - 반출 시각 - 현재 사용자 이름

예시 문구 - `에어컨 세척기 반출 완료`

5-5. 반출 실패 케이스

이미 반출 중

- 메시지: `이미 반출 중인 장비입니다.`
- 표시: 현재 보유자 / 반출 시각

비활성 장비

- 메시지: `현재 사용 중지된 장비입니다.`

태그 불일치

- 메시지: `장비 태그 정보가 일치하지 않습니다.`

트랜잭션 충돌

- 메시지: `동시에 처리 중입니다. 다시 시도해 주세요.`

---

6. 반입 처리 엔진

6-1. 반입 입력값

- actorUserId
- nfcTagId
- equipmentId
- note
- clientTimestamp(optional)

6-2. 반입 전 검증

- 사용자 ACTIVE
- equipment exists
- isActive = true
- status = CHECKED_OUT 또는 OVERDUE
- nfcTagId 일치

6-3. 반입 트랜잭션 처리

1. equipment 문서 재조회
2. status가 CHECKED_OUT 또는 OVERDUE인지 확인
3. status를 AVAILABLE로 변경
4. currentHolderUserId 제거
5. currentHolderName 제거
6. lastCheckinAt = serverTimestamp
7. equipment_logs 문서 생성
8. audit_logs 필요 시 생성
9. 커밋

6-4. 반입 성공 결과

UI 표시 - 장비명 - 반입 성공 메시지 - 반입 시각

예시 문구 - `에어컨 세척기 반입 완료`

6-5. 반입 실패 케이스

이미 보관 상태

- 메시지: `이미 보관 상태인 장비입니다.`

비활성 장비

- 메시지: `현재 사용 중지된 장비입니다.`

트랜잭션 충돌

- 메시지: `동시에 처리 중입니다. 다시 시도해 주세요.`

---

7. 미반입 자동 판정 엔진

7-1. 목적

반출 후 일정 기준을 넘겨도 반입되지 않은 장비를 자동으로 식별한다.

7-2. 판정 기준

기본 예시 - lastCheckoutAt 기준 24시간 초과

정책 확장 가능 - 장비 유형별 다른 기준 - 사업분야별 다른 기준

7-3. 판정 방식

방식 A. 조회 시 계산

목록 조회 시 - status = CHECKED_OUT - now - lastCheckoutAt > threshold 이면 UI에서 overdue 표시

장점 - 구현 간단 단점 - 상태 일관성 약함

방식 B. 배치 또는 트리거 갱신

스케줄러 또는 관리자 조회 시 - 조건 충족 시 status = OVERDUE 전환

장점 - 상태 명확 권장 - **MVP는 A+B 혼합** - UI 계산 우선 - 추후 배치 확정

7-4. 관리자 표시 규칙

- 대시보드 KPI 반영
- 미반입 목록 상단 배치
- 오래된 순 정렬

- 경고 색상 강조

---

8. 동시성 제어 설계

8-1. 문제 상황

예시 - 두 명이 같은 장비를 거의 동시에 태깅 - 한 명은 반출, 다른 한 명은 반입 시도 - 네트워크 지연으로 이전 상태 기준 처리 시도

8-2. 해결 원칙

- 모든 상태 변경은 Firestore transaction 사용
- UI 상태를 신뢰하지 말고 서버 최신 상태 재확인
- 트랜잭션 실패 시 사용자에게 재시도 안내

8-3. 중복 요청 방지

클라이언트 - 버튼 연타 방지 - isSubmitting 상태

서버 - status 재검증 - 동일 장비 상태 전이 중복 방지

---

9. 로그 설계

9-1. equipment_logs 기록 원칙

모든 반출/반입/강제 상태변경은 로그 남김

9-2. actionType enum

- REGISTER
- NFC_MAP
- CHECK_OUT
- CHECK_IN
- FORCE_UPDATE
- DISABLE

9-3. CHECK_OUT 로그 예시

```
{
  "equipmentId": "eq_001",
  "equipmentCode": "EQ-AC-001",
  "equipmentName": "에어컨 세척기",
  "nfcTagId": "04AABBCC1122",
  "actionType": "CHECK_OUT",
```

```
  "actorUserId": "uid_123",
  "actorUserName": "홍길동",
  "reason": "현장 작업",
  "useLocation": "해운대 상가",
  "note": "정상 반출",
  "result": "SUCCESS",
  "performedAt": "serverTimestamp"
}
```

9-4. CHECK_IN 로그 예시

```
{
  "equipmentId": "eq_001",
  "actionType": "CHECK_IN",
  "actorUserId": "uid_123",
  "actorUserName": "홍길동",
  "note": "정상 반입",
  "result": "SUCCESS",
  "performedAt": "serverTimestamp"
}
```

---

10. API 구조 설계

10-1. 장비 조회 API

`getEquipmentByNfcTag(nfcTagId)`

역할 - 태그 기반 장비 조회

`getEquipmentDetail(equipmentId)`

역할 - 장비 상세 조회

`getEquipmentLogs(filters)`

역할 - 장비 로그 조회

10-2. NFC 처리 API

`processNfcScan(nfcTagId, actorUserId, mode)`

역할 - 스캔 결과 기반 장비/상태 조회 - 반출/반입 가능성 판단

반환 예시

```
interface ProcessNfcScanResult {
  equipmentId: string;
  equipmentName: string;
  currentStatus: 'AVAILABLE' | 'CHECKED_OUT' | 'OVERDUE' | 'DISABLED';
  allowedAction: 'CHECK_OUT' | 'CHECK_IN' | 'NONE';
  message?: string;
}
```

10-3. 반출/반입 실행 API

`checkoutEquipment(payload)`

역할 - 반출 트랜잭션 실행

`checkinEquipment(payload)`

역할 - 반입 트랜잭션 실행

10-4. 관리자 강제 처리 API

`forceUpdateEquipmentStatus(payload)`

역할 - 관리자 예외 복구

필수 요구 - reason 필수 - audit_logs 필수

---

11. 실시간 처리 설계

11-1. 목적

관리자 대시보드/장비 목록에서 장비 상태가 빠르게 반영되게 한다.

11-2. 처리 방식

- Firestore onSnapshot 실시간 구독 가능
- 최소 MVP는 조회 기반 + 수동 새로고침도 가능
- 관리자 장비 목록은 실시간 구독 권장

11-3. 실시간 반영 대상

- equipments.status
- equipments.currentHolderName
- equipments.lastCheckoutAt

- equipment_logs recent list

---

12. 화면 모드별 UX 설계

12-1. 사원 반출 화면

흐름 1. 반출 모드 진입 2. NFC 태깅 3. 장비 정보 표시 4. 반출 사유/위치 입력 5. 반출 확인 6. 성공 메시지

12-2. 사원 반입 화면

흐름 1. 반입 모드 진입 2. NFC 태깅 3. 장비 정보 표시 4. 현재 보유자/반출시각 표시 5. 반입 메모 입력 6. 반입 확인

12-3. 관리자 장비 목록 화면

표시 - 상태 - 현재 사용자 - 미반입 여부 - 최근 로그

---

13. 권한 정책 연동

EMPLOYEE

허용 - 본인 반출 - 본인 반입 - 본인 로그 조회(제한)

금지 - 장비 등록 - NFC 매핑 - 강제 상태 변경

LEADER

기본 - EMPLOYEE와 동일 확장 가능 - 팀 범위 장비 조회

ADMIN

허용 - 장비 등록/수정 - NFC 매핑 - 전체 로그 조회 - 미반입 관리 - 강제 상태 복구

SUPER_ADMIN

- ADMIN과 동일 + 시스템 정책 변경

---

14. 예외 처리 정책

14-1. 태그 미등록

- 사원: 차단

- 관리자: 등록 화면 이동 유도 가능

14-2. 중복 ACTIVE 매핑

- 시스템 오류
- 처리 차단
- 관리자 경고

14-3. 반출 중 장비 반출 시도

- 차단
- 현재 보유자 정보 노출

14-4. 반입 시 보관 상태 장비

- 차단
- 사용자 안내

14-5. 네트워크 실패

- 처리 전 실패: 재시도
- 처리 중 실패: 결과 재조회 유도

14-6. 서버 응답 지연

- 버튼 로딩 유지
- 중복 클릭 차단

---

15. 감사(Audit) 정책

필수 감사 대상

- 장비 등록
- NFC 매핑
- 관리자 강제 상태 변경
- 장비 비활성화
- 미반입 예외 복구

audit_logs 필드 예시

- actorUserId
- actorUserName
- targetCollection
- targetDocumentId
- action
- beforeData
- afterData
- reason

- createdAt

---

16. 확장 설계 포인트

16-1. QR fallback

NFC 미지원 기기용 QR 코드 백업 흐름 추가 가능

16-2. 장비 유형별 정책

- 공용 장비
- 개인 전용 장비
- 현장 전용 장비

16-3. 다지점/가맹점 확장

- organizationId
- branchId 필드 추가로 분리 가능

16-4. 예약 반출

향후 일정과 연결된 사전 예약 반출 기능 가능

16-5. 실시간 알림

미반입 임계치 초과 시 관리자 알림 가능

---

17. 테스트 시나리오

17-1. 정상 반출

1. AVAILABLE 장비 태깅
2. 반출 모드
3. reason 입력
4. 반출 성공
5. status = CHECKED_OUT

17-2. 정상 반입

1. CHECKED_OUT 장비 태깅
2. 반입 모드
3. 반입 성공
4. status = AVAILABLE

17-3. 동시 반출 충돌

1. 두 사용자가 같은 장비 태깅
2. 한 요청만 성공
3. 나머지는 충돌 메시지

17-4. 미반입 판정

1. 반출 후 기준 시간 초과
2. 관리자 화면 overdue 표시

17-5. 태그 미등록

1. 미등록 태그 태깅
2. 사원 차단
3. 관리자 등록 유도

---

18. 구현 우선순위

18-1. 1차 필수

- NFC UID 읽기
- 태그→장비 매핑 조회
- 반출 트랜잭션
- 반입 트랜잭션
- 장비 상태 머신
- 로그 저장
- 관리자 장비 목록 반영

18-2. 2차 개선

- OVERDUE 자동 배치
- 실시간 구독 고도화
- QR fallback
- 알림 기능

---

19. 최종 정리

이 문서는 HELLA company 업무관리사이트의 NFC 반출/반입 기능을 실제 엔진 수준으로 설계한 문서다.

이 문서를 기준으로 다음 작업을 바로 진행할 수 있다. 1. NFC 처리 서비스 코드 작성 2. Firestore transaction 구현 3. 장비 상태 UI 반영 4. 로그 및 감사 정책 구현 5. 관리자 미반입 모니터링 구현

다음으로 가장 자연스럽게 이어질 문서는 - **HELLA company 업무관리사이트 — Firestore Rules 설계서** 또는 - **HELLA company 업무관리사이트 — API 명세서**